PC Hardware Marketplace

System Architecture & Feature Documentation

PC Hardware Marketplace

System Architecture & Feature Documentation

Platform Name: PCForge
Repository: `pc-hardware-marketplace`
Document Version: 1.0
Prepared On: March 12, 2026
Document Scope: Current implemented state of the full codebase

This document describes the website as it exists today across the frontend, backend, database layer, infrastructure, marketplace logic, hardware reference data, and operational tooling.

Table of Contents

1. Executive Summary
2. Platform Purpose
3. Current Feature Set
4. User Flow
5. Frontend Architecture
6. Backend Architecture
7. Database Design
8. Hardware Reference Database
9. Infrastructure
10. Current Limitations
11. Technical Debt
12. MVP Evaluation
13. Full System Diagram
14. Conclusion

1. Executive Summary

PCForge is a PC hardware marketplace focused on buying and selling used PC components, with the strongest implementation currently centered on GPU marketplace listings. The platform also includes a secondary product catalog for non-GPU components, a basic PC builder, lightweight price tracking, wishlisting, cart and checkout flows, seller analytics, and admin demo-seeding tools.

From a product perspective, the application already supports the core buyer loop of discovery, detail view, cart, checkout, and order persistence. From an engineering perspective, the system is a containerized full-stack application built with:

- Next.js on the frontend
- NestJS on the backend
- PostgreSQL for primary persistence
- Redis for limited caching
- Elasticsearch for partial indexing
- Nginx for reverse proxying
- Docker Compose for local orchestration

The most important architectural fact about the current implementation is that the marketplace is represented by **two overlapping inventory systems**:

- A general `Product` catalog used for CPUs, RAM, storage, motherboards, PSUs, coolers, cases, fans, and accessories
- A GPU-specific `Listing` marketplace that is normalized against a canonical `Gpu` reference database

The frontend merges these two models into a single browse experience, but they are not actually unified in the backend or schema design. That split shapes nearly every current feature, limitation, and technical debt item in the project.

2. Platform Purpose

2.1 Problem the Platform Solves

The platform is designed to solve common problems in used PC hardware commerce:

- Buyers need a structured way to discover components instead of relying on inconsistent marketplace titles and unstructured listings.
- Sellers need a way to publish inventory with technical metadata that buyers care about.
- Builders need compatibility guidance before purchasing multiple parts.
- Marketplace operators need internal tooling to seed inventory, inspect coverage, and eventually moderate sellers and listings.

2.2 Intended User Types

The codebase currently represents three primary user types:

- **Buyers**
  - Browse inventory
  - Search and filter listings
  - Save listings to a wishlist
  - Add products and listings to cart
  - Save addresses and payment methods
  - Place orders
- **Sellers**
  - Create GPU listings
  - Create non-GPU products
  - Review listing analytics
  - Edit or delete GPU marketplace listings
- **Admins**
  - Seed demo product catalog data
  - Seed demo GPU marketplace listings
  - Remove seeded inventory
  - View high-level platform inventory summaries

2.3 Current Product Positioning

As implemented today, the product is best understood as:

- A **used PC hardware marketplace prototype** with real buyer-facing flows
- A **GPU-first marketplace** with stronger normalization and browse quality on the GPU side
- A **partially generalized catalog** for other hardware categories
- A **secondary PC builder** that depends on marketplace data but is not the primary product

3. Current Feature Set

3.1 Feature Inventory Matrix

Feature Area	Status	Summary	Primary Implementation
Landing page and integrated marketplace feed	Partial	Marketing surface plus live inventory feed	`frontend/src/pages/index.tsx`, `frontend/src/components/marketplace/MarketplaceCatalogFeed.tsx`
Alias browse routes	Complete	`/browse`, `/components`, and `/listings` all route to the same	`frontend/src/pages/browse.tsx`, `frontend/src/pages/components.tsx`, `frontend/src/pages/listings/index.tsx`

		discovery experience	
General product catalog	Partial	Non-GPU hardware stored as `Product` records with JSON specs and compatibility metadata	`backend/src/modules/products/*`, `frontend/src/components/ProductCard.tsx`
GPU marketplace listings	Mostly complete	Canonical GPU-backed listings with images, views, saves, cart, and seller profile links	`backend/src/listings/*`, `frontend/src/pages/listings/[id].tsx`
GPU reference database	Complete for GPUs	Canonical GPU records seeded from CSV	`backend/src/gpus/*`, `backend/database/seed/gpu_seed_series.csv`
Search	Partial	Frontend search and search routes exist, but user-facing search remains mostly SQL-backed	`backend/src/modules/search/*`, `frontend/src/components/ui/FilterSidebar.tsx`
Filtering	Partial	Marketplace feed supports category, price, rating, and stock filters, but not all filters apply to all inventory types	`frontend/src/components/marketplace/MarketplaceCatalogFeed.tsx`
Compare	Partial	Product comparison exists for `Product` records only	`frontend/src/store/useCompareStore.ts`, `frontend/src/components/CompareTable.tsx`
Product price history	Partial	Product prices are tracked and charted	`backend/src/modules/products/entities/price-history.entity.ts`, `frontend/src/components/PriceHistoryChart.tsx`

Listing price history	Partial	Listing price history is persisted in the backend but not fully surfaced in the UI	`backend/src/listings/entities/listing-price-history.entity.ts`
Authentication	Complete for basic auth	JWT registration, login, current-user bootstrap, role checks	`backend/src/modules/auth/*`, `frontend/src/components/AuthBootstrap.tsx`
Seller accounts	Partial	Seller entity, seller status, seller analytics, and seller dashboard shell exist	`backend/src/modules/sellers/*`, `frontend/src/pages/dashboard/seller.tsx`
Public seller profile	Complete for marketplace listings	Buyers can browse a seller's live GPU listings	`frontend/src/pages/sellers/[id].tsx`, `backend/src/listings/listings.service.ts`
Wishlist	Complete for listings only	Saved GPU listings per user	`backend/src/listings/wishlist.service.ts`, `frontend/src/pages/wishlist.tsx`
Cart	Complete	Supports both products and GPU listings	`backend/src/modules/cart/*`, `frontend/src/pages/cart.tsx`
Checkout and orders	Partial	Order rows and order items are persisted, but payment and fulfillment are not real	`backend/src/modules/orders/*`, `frontend/src/pages/checkout.tsx`
Addresses	Complete	Saved address management with default selection	`backend/src/modules/addresses/*`, `frontend/src/pages/addresses.tsx`
Payment methods	Partial	Flexible stored payment metadata with provider and type selection	`backend/src/modules/payment-methods/*`, `frontend/src/pages/payment-methods.tsx`

Reviews	Partial	Backend exists, frontend is not connected	`backend/src/modules/reviews/*`
PC builder	Partial	Client-side builder with backend compatibility validation	`backend/src/modules/pc-builder/*`, `frontend/src/pages/pc-builder.tsx`
Admin panel	Partial	Platform overview plus demo-seeding and removal tools	`frontend/src/pages/dashboard/admin.tsx`
Deals page	Partial / placeholder	Uses first six products from live product feed, not real deal logic	`frontend/src/pages/deals.tsx`
Static support and policy pages	Complete	Help, contact, trust and safety, privacy, terms, cookies	`frontend/src/pages/help.tsx`, `contact.tsx`, `trust-safety.tsx`, `privacy.tsx`, `terms.tsx`, `cookies.tsx`
Redis caching	Partial	One latest-products cache key	`backend/src/modules/products/products.service.ts`
Elasticsearch indexing	Partial	Product indexing exists, but interactive search does not depend on Elasticsearch	`backend/src/modules/search/*`, `backend/src/modules/products/products.service.ts`

3.2 Marketplace Discovery and Navigation

The landing page is the primary browse surface and combines:

- A hero section
- Category shortcuts
- Featured GPU listing cards
- An embedded marketplace feed

Key files:

- `frontend/src/pages/index.tsx`
- `frontend/src/components/Header.tsx`
- `frontend/src/components/Footer.tsx`
- `frontend/src/components/marketplace/MarketplaceCatalogFeed.tsx`

Technical behavior:

- Header search submits to `/?q=<query>#marketplace`
- Category tiles append `?category=<CATEGORY>#marketplace`
- The feed renders both products and GPU listings in one grid
- Listing cards only render when category is unset or `GPU`

Why the feature is partial:

- Category counts are only real for GPU inventory
- Non-GPU coverage is implied more strongly than it is implemented
- Multiple route aliases point to the same page

3.3 General Product Catalog

The `Product` model is the platform's generic hardware catalog entity.

Key files:

- `backend/src/modules/products/entities/product.entity.ts`
- `backend/src/modules/products/products.service.ts`
- `backend/src/modules/products/products.controller.ts`
- `frontend/src/features/products.ts`
- `frontend/src/components/ProductCard.tsx`
- `frontend/src/pages/products/[id].tsx`

`Product` supports:

- Name
- Brand
- Category
- Price
- Images
- JSONB `specs`
- JSONB `compatibility`
- Stock state
- Rating
- Rating count
- Seller relation
- Price history relation
- Review relation

How it works technically:

- `GET /products` returns latest products
- `GET /search/products` executes SQL/TypeORM filtering
- `GET /products/:id` loads `priceHistory` and `reviews`
- `POST /products` creates a seller-owned product
- `PUT /products/:id` updates a product with owner or admin enforcement
- `DELETE /products/:id` is admin-only
- Demo catalog products can be created through `POST /products/admin/demo-catalog`

Current completion status: **Partial**

Reasons:

- There is no complete seller product-management UI equivalent to listing management
- Product wishlisting does not exist
- Public seller profile is not tied to product sellers
- Frontend review interfaces are missing

3.4 GPU Marketplace Listings

GPU listings are the strongest marketplace implementation in the codebase.

Key files:

- `backend/src/listings/entities/listing.entity.ts`
- `backend/src/listings/listings.service.ts`
- `backend/src/listings/listings.controller.ts`
- `backend/src/listings/wishlist.service.ts`
- `frontend/src/components/ui/ListingCard.tsx`
- `frontend/src/pages/listings/[id].tsx`
- `frontend/src/pages/profile/my-listings.tsx`
- `frontend/src/pages/sellers/[id].tsx`

Current listing capabilities:

- Browse GPU listings
- Search GPU listings
- Filter GPU listings by text and price
- View listing detail pages
- View seller profile
- Save to wishlist
- Add to cart
- Buy now
- Create new listing
- Edit listing price and condition
- Delete owned listing

- Track listing views and wishlist saves

How it works technically:

- Every listing belongs to a canonical `Gpu` record
- Every listing belongs to a mirrored marketplace user row
- Listing images are stored in `listing_images`
- Listing views are tracked in `listing_views`
- Wishlist rows are stored in `wishlists`
- Manual listing price changes are stored in `listing_price_history`
- Demo listing seeding creates realistic inventory from the GPU reference database

Current completion status: **Mostly complete for GPU marketplace flows**

Remaining gaps:

- Only GPUs use this normalized marketplace model
- Listing detail pages do not show the stored listing price-history chart
- There is no seller messaging or negotiation flow

3.5 GPU Hardware Reference Database

The platform includes a real hardware-reference subsystem for GPUs.

Key files:

- `backend/src/gpus/gpus.service.ts`
- `backend/src/gpus/gpus.controller.ts`
- `backend/src/gpus/entities/gpu.entity.ts`
- `backend/src/gpus/entities/gpu-price-history.entity.ts`
- `backend/database/seed/gpu_seed_series.csv`
- `backend/database/seed/seed-gpus.js`

Reference data currently includes:

- 79 GPU rows total
- 49 NVIDIA rows
- 23 AMD rows
- 7 Intel rows

Stored reference fields:

- `name`
- `slug`
- `manufacturer`
- `architecture`
- `releaseYear`
- `processNm`

- `vramGb`
- `memoryType`
- `memoryBusWidth`
- `pcieVersion`
- `tdpWatts`

Current completion status: **Complete for GPUs only**

3.6 Search

Search exists at both the API and UI layers.

Key files:

- `backend/src/modules/search/search.controller.ts`
- `backend/src/modules/search/search.service.ts`
- `frontend/src/components/Header.tsx`
- `frontend/src/components/ui/FilterSidebar.tsx`
- `frontend/src/hooks/useProducts.ts`
- `frontend/src/hooks/useListings.ts`

API search routes:

```
GET /api/search/health
GET /api/search/products
GET /api/search/listings
GET /api/search/gpus
GET /api/search/all
```

How it works technically:

- Product search uses SQL with ILIKE filters
- Listing search uses SQL over joined GPU fields
- GPU reference search uses SQL over GPU rows
- `searchAll` aggregates all three
- Product indexing exists in Elasticsearch, but the search service delegates back to SQL-backed services

Current completion status: **Partial**

3.7 Filtering and Catalog Controls

The marketplace feed includes shared filters for:

- Text search
- Category
- Minimum price
- Maximum price

- Minimum rating
- In-stock only

Key files:

- `frontend/src/components/ui/FilterSidebar.tsx`
- `frontend/src/components/ProductFilters.tsx`
- `frontend/src/components/marketplace/MarketplaceCatalogFeed.tsx`

Current completion status: **Partial**

Reason:

- Product filters are richer than listing filters
- Category and rating semantics do not apply uniformly across both inventory models

3.8 Comparison

Product comparison is implemented as a lightweight client-side tool.

Key files:

- `frontend/src/store/useCompareStore.ts`
- `frontend/src/components/ProductCard.tsx`
- `frontend/src/components/CompareTable.tsx`

How it works:

- Users toggle product IDs into a Zustand store
- The marketplace feed filters selected products
- A comparison table renders brand, price, and rating

Current completion status: **Partial**

Reason:

- Listings cannot be compared
- Deep spec comparison is not implemented

3.9 Price Tracking

Price tracking exists in three forms:

- `price_history` for `Product`
- `gpu_price_history` for GPU reference and seeded listing snapshots
- `listing_price_history` for explicit listing price changes

Key files:

- `backend/src/modules/products/entities/price-history.entity.ts`
- `backend/src/gpus/entities/gpu-price-history.entity.ts`

- `backend/src/listings/entities/listing-price-history.entity.ts`
- `frontend/src/components/PriceHistoryChart.tsx`
- `frontend/src/pages/products/[id].tsx`

How it works:

- Product history rows are written on create and price change
- Product detail pages show a line chart
- Listing price changes write the old price to `listing_price_history`
- Listing creation and demo seeding also write a `gpu_price_history` entry

Current completion status: **Partial**

3.10 Authentication

The authentication system is JWT-based and works end-to-end.

Key files:

- `backend/src/modules/users/users.controller.ts`
- `backend/src/modules/users/users.service.ts`
- `backend/src/modules/auth/auth.service.ts`
- `backend/src/modules/auth/jwt.strategy.ts`
- `backend/src/modules/auth/jwt.guard.ts`
- `frontend/src/features/auth.ts`
- `frontend/src/store/useUserStore.ts`
- `frontend/src/components/AuthBootstrap.tsx`

How it works:

- Registration creates a user with a bcrypt password hash
- Login validates email and password
- Tokens encode `sub`, `email`, and `role`
- Frontend stores token in `localStorage`
- Axios attaches bearer token automatically
- `AuthBootstrap` fetches `/users/me` to populate the user store

Current completion status: **Complete for basic authentication**

What is missing:

- Password reset
- Email verification
- Refresh tokens
- Secure cookie sessions
- Session management and logout across devices

3.11 Seller Accounts and Seller Analytics

The platform includes a dedicated seller module.

Key files:

- `backend/src/modules/sellers/entities/seller.entity.ts`
- `backend/src/modules/sellers/sellers.service.ts`
- `backend/src/modules/sellers/sellers.controller.ts`
- `frontend/src/pages/dashboard/seller.tsx`
- `frontend/src/features/sellers.ts`

Current backend capabilities:

- Apply as seller
- List seller rows for the current user
- Update seller status as admin
- View seller analytics

Important behavior:

- `ensureApprovedSellerForUser` auto-approves a seller if one already exists but is not approved
- If no seller exists, it creates one automatically and marks it approved

Current completion status: **Partial and internally inconsistent**

Reason:

- Product sellers and GPU listing sellers are modeled differently
- Seller analytics only count products, not marketplace GPU listings
- There is no frontend seller application flow

3.12 Public Seller Profiles

Public seller profiles are implemented for marketplace listing sellers.

Key files:

- `frontend/src/pages/sellers/[id].tsx`
- `backend/src/listings/listings.service.ts`

How it works:

- Seller profile data is loaded from `GET /listings/sellers/:id/profile`
- The profile shows seller name, role, joined date, total listings, and listing cards

Current completion status: **Complete for listing sellers only**

3.13 Wishlist

Wishlist functionality exists for GPU listings.

Key files:

- `backend/src/listings/wishlist.entity.ts`
- `backend/src/listings/wishlist.service.ts`
- `backend/src/listings/wishlist.controller.ts`
- `frontend/src/features/wishlist.ts`
- `frontend/src/pages/wishlist.tsx`

How it works:

- Logged-in users can save a listing from listing detail
- Wishlist rows are unique on `(user, listing)`
- The wishlist page renders saved listing cards

Current completion status: **Complete for listings only**

3.14 Cart

The cart supports mixed inventory.

Key files:

- `backend/src/modules/cart/entities/cart-item.entity.ts`
- `backend/src/modules/cart/cart.service.ts`
- `backend/src/modules/cart/cart.controller.ts`
- `frontend/src/features/cart.ts`
- `frontend/src/pages/cart.tsx`

What the cart supports:

- `PRODUCT` items
- `LISTING` items
- Quantity changes
- Removal
- Clear cart
- Snapshot persistence for display resilience

Current completion status: **Complete**

3.15 Checkout and Orders

Checkout and order persistence are real, but still limited.

Key files:

- `backend/src/modules/orders/entities/order.entity.ts`
- `backend/src/modules/orders/entities/order-item.entity.ts`
- `backend/src/modules/orders/orders.service.ts`
- `backend/src/modules/orders/orders.controller.ts`
- `frontend/src/features/orders.ts`

- `frontend/src/pages/checkout.tsx`
- `frontend/src/pages/orders.tsx`

How it works:

- Checkout loads cart, addresses, and payment methods
- Users provide shipping and tax inputs manually
- The backend loads referenced products and listings
- Order items are stored with snapshots
- Orders can optionally clear the cart after creation

Current completion status: **Partial**

Major missing pieces:

- No payment gateway
- No inventory reservation
- No seller-side fulfillment workflow
- No transaction wrapping for race-condition safety
- No order status lifecycle beyond persisted enum fields

3.16 Addresses

Address management is one of the most complete account sub-features.

Key files:

- `backend/src/modules/addresses/addresses.service.ts`
- `backend/src/modules/addresses/addresses.controller.ts`
- `frontend/src/pages/addresses.tsx`

Capabilities:

- Create address
- List addresses
- Delete address
- Mark address as default

Current completion status: **Complete**

3.17 Payment Methods

Payment methods are modeled flexibly in the backend and surfaced through a dynamic form in the frontend.

Key files:

- `backend/src/modules/payment-methods/payment-methods.service.ts`
- `backend/src/modules/payment-methods/payment-methods.controller.ts`
- `backend/src/modules/payment-methods/entities/payment-method.entity.ts`
- `frontend/src/pages/payment-methods.tsx`

Supported payment types:

- CARD
- UPI
- NET_BANKING
- WALLET
- COD
- BANK_TRANSFER

Current completion status: **Partial**

Reason:

- These rows are saved metadata, not integrated payment instruments
- Checkout does not process real payments

3.18 Reviews

The review subsystem exists only at the backend layer.

Key files:

- `backend/src/modules/reviews/reviews.service.ts`
- `backend/src/modules/reviews/reviews.controller.ts`
- `backend/src/modules/reviews/entities/review.entity.ts`

Capabilities:

- List reviews for a product
- Create a review for a product
- Recalculate product rating and rating count

Current completion status: **Partial**

Reason:

- No review creation UI
- No review list UI

3.19 PC Builder

The PC builder is implemented as a secondary discovery tool.

Key files:

- `backend/src/modules/pc-builder/pc-builder.service.ts`
- `backend/src/modules/pc-builder/pc-builder.controller.ts`
- `frontend/src/pages/pc-builder.tsx`
- `frontend/src/store/usePcBuilderStore.ts`
- `frontend/src/components/PcBuilder/PartSelector.tsx`

- `frontend/src/components/PcBuilder/PcBuilderSummary.tsx`

Validation rules currently implemented:

- CPU socket vs motherboard socket
- RAM type vs motherboard RAM type
- GPU length vs case GPU clearance
- PSU wattage vs estimated CPU/GPU power need

Current completion status: **Partial**

Reason:

- Compatibility logic is basic
- The builder uses only `Product` rows, not marketplace listings
- It depends entirely on seller-entered compatibility JSON for non-GPU categories

3.20 Admin Panel

The admin panel is an operational surface for inventory seeding and summary metrics.

Key files:

- `frontend/src/pages/dashboard/admin.tsx`
- `backend/src/modules/products/products.service.ts`
- `backend/src/listings/listings.service.ts`

Current admin actions:

- Seed demo products
- Seed demo GPU listings
- Delete demo products
- Delete admin-owned listings
- View counts of products, listings, and unique listing sellers

Current completion status: **Partial**

Missing admin workflows:

- Seller approval queue UI
- User management
- Moderation tools
- Platform configuration

3.21 Deals Page

The deals page is not a true pricing engine.

Key file:

- `frontend/src/pages/deals.tsx`

Technical behavior:

- Fetches all products
- Displays the first six as "deals"

Current completion status: **Partial / placeholder**

3.22 Static Support and Policy Pages

The platform includes static pages for:

- Help center
- Contact
- Trust and safety
- Privacy policy
- Terms of use
- Cookies

Key files:

- `frontend/src/pages/help.tsx`
- `frontend/src/pages/contact.tsx`
- `frontend/src/pages/trust-safety.tsx`
- `frontend/src/pages/privacy.tsx`
- `frontend/src/pages/terms.tsx`
- `frontend/src/pages/cookies.tsx`

Current completion status: **Complete**

3.23 Redis Caching

Redis is wired into the backend but used narrowly.

Key files:

- `backend/src/config/redis.config.ts`
- `backend/src/modules/products/products.module.ts`
- `backend/src/modules/products/products.service.ts`

Current behavior:

- `GET /products` caches latest results under `products:featured`
- Cache TTL is 60 seconds
- Cache is invalidated on product create, update, and demo seed

Current completion status: **Partial**

3.24 Elasticsearch Indexing

Elasticsearch exists as deployed infrastructure and product-indexing code.

Key files:

- `backend/src/config/elastic.config.ts`
- `backend/src/modules/search/search.module.ts`
- `backend/src/modules/search/search.service.ts`
- `backend/src/modules/products/products.service.ts`

Current behavior:

- Product create and update index documents
- Product delete attempts to remove the document
- Search health endpoint returns cluster health
- Mapping bootstrap logic exists in `ensureIndex()`

Current completion status: **Partial**

Reason:

- Interactive user search does not actually depend on Elasticsearch queries

4. User Flow

4.1 Buyer Flow

Step 1: Land on the Website

The buyer opens `/`, which renders:

- Marketplace-oriented branding
- Search bar
- Category shortcuts
- Featured listing cards
- Integrated marketplace feed

Implementation:

- `frontend/src/pages/index.tsx`
- `frontend/src/components/Header.tsx`
- `frontend/src/components/marketplace/MarketplaceCatalogFeed.tsx`

Step 2: Search for a Component

The buyer uses the header search bar.

Technical sequence:

1. Search form submits in `Header.tsx`
2. Frontend navigates to `/?q=<search>#marketplace`
3. `MarketplaceCatalogFeed` reads the query from router state

4. Product query runs through `useProducts`
5. Listing query runs through `useListings`
6. Backend search routes return results

Step 3: Filter Results

The buyer adjusts filters in the left sidebar.

Technical sequence:

1. `FilterSidebar` updates local filter state
2. `MarketplaceCatalogFeed` computes product filters and listing filters separately
3. React Query re-runs the relevant queries
4. Results render as a mixed grid of `ProductCard` and `ListingCard`

Step 4: Open a Listing or Product

There are two detail-page flows:

- `Product` detail: `/products/:id`
- `Listing` detail: `/listings/:id`

Product detail behavior:

- Loads product by UUID
- Shows brand, seller name, stock, rating, specs, price history
- Supports add-to-cart and buy-now for authenticated users

Listing detail behavior:

- Loads listing by numeric ID
- Tracks a view row
- Shows canonical GPU reference specs
- Links to public seller profile
- Supports wishlist, add-to-cart, and buy-now

Step 5: Save, Compare, or Buy

Available buyer actions:

- Save GPU listing to wishlist
- Add product to compare table
- Add product or listing to cart
- Buy now by pushing item into cart and redirecting to checkout

Step 6: Authenticate

If the buyer is not logged in:

- `/login` or `/register` is used
- JWT is stored in `localStorage`

- `AuthBootstrap` fetches `/users/me`
- User role and navigation state are populated

Step 7: Checkout

Checkout flow:

1. Buyer opens `/checkout`
2. Frontend fetches cart, addresses, payment methods
3. Buyer selects address and payment method
4. Buyer sets shipping and tax amounts
5. Frontend posts to `/orders/checkout`
6. Backend creates `Order` and `OrderItem` rows
7. Cart can be cleared automatically

Step 8: View Orders

The buyer opens `/orders` to review:

- Order count
- Total spend
- Latest status
- Order line items

4.2 Seller Flow

Step 1: Seller Access

Seller access is role-based. The frontend checks whether the user is `SELLER` or `ADMIN`.

Key surfaces:

- Header seller link
- Profile menu seller link
- `/sell`
- `/dashboard/seller`
- `/profile/my-listings`

Step 2: Post Inventory

The sell page is split by category.

GPU path

1. Search GPU reference records through `/search/gpus`
2. Select canonical GPU
3. Enter price, condition, description, image URLs
4. Submit to `/listings`

Non-GPU path

1. Choose category
2. Enter brand and name
3. Fill category-specific specs
4. Fill compatibility fields
5. Submit to `/products`

Step 3: Review Seller Dashboard

`/dashboard/seller` loads seller analytics from `/sellers/me/analytics`.

Displayed metrics:

- Total products
- Total sales
- Revenue

Current caveat:

- Total sales and revenue are placeholders
- Product totals do not reflect GPU listing inventory

Step 4: Manage Marketplace Listings

`/profile/my-listings` is the real seller management page for GPU listings.

Capabilities:

- Review views and saves
- Edit price
- Edit condition
- Delete listing

4.3 Admin Flow

Admins can open `/dashboard/admin`.

Current actions:

- Seed demo product catalog data
- Seed demo GPU listings
- Delete seeded products
- Delete admin-owned listings
- Review platform inventory coverage

5. Frontend Architecture

5.1 Framework and Runtime

Frontend framework:

- **Next.js 14**
- **Pages Router**
- **React 18**

Key files:

- `frontend/package.json`
- `frontend/src/pages/_app.tsx`
- `frontend/next.config.mjs`

Core runtime libraries:

- `@tanstack/react-query`
- `axios`
- `zustand`
- `tailwindcss`

5.2 Frontend Application Bootstrap

The frontend boot path is:

1. Next.js mounts `_app.tsx`
2. React Query `QueryClientProvider` is created
3. Google fonts are loaded
4. `AuthBootstrap` hydrates the token and fetches current user
5. Page components render inside shared layout wrappers

Key file:

- `frontend/src/pages/_app.tsx`

5.3 Layout and Visual Structure

Layout stack:

- `Layout.tsx`
- `PageLayout.tsx`
- `SiteLayout.tsx`
- `Header.tsx`
- `Footer.tsx`
- `BackgroundGrid.tsx`

Purpose of each layer:

Component	Responsibility
`Layout.tsx`	Thin wrapper around page layout
`PageLayout.tsx`	Layout handoff and page shell

`SiteLayout.tsx`	Header, footer, main content region
`Header.tsx`	Global navigation and search
`Footer.tsx`	Global footer links
`BackgroundGrid.tsx`	Visual atmospheric background

5.4 Styling and Design System

Styling is built from Tailwind plus custom CSS utility classes.

Key files:

- `frontend/src/styles/globals.css`
- `frontend/src/styles/theme.ts`
- `frontend/tailwind.config.ts`

Design characteristics:

- Dark marketplace theme
- Indigo / violet accent palette
- Custom utility classes such as:
  - `pf-card`
  - `pf-input`
  - `pf-button-primary`
  - `pf-button-secondary`
  - `pf-badge`

5.5 Routing Structure

Primary page routes:

Route	Purpose
`/`	Landing page plus integrated marketplace feed
`/browse`	Alias of home page
`/components`	Alias of home page
`/listings`	Alias of home page
`/products/[id]`	General product detail
`/listings/[id]`	GPU marketplace listing detail
`/sellers/[id]`	Public seller profile
`/sell`	Seller inventory creation flow
`/cart`	Cart management

`/checkout`	Checkout
`/orders`	Order history
`/wishlist`	Saved GPU listings
`/profile`	Account hub
`/profile/my-listings`	Listing owner analytics and edits
`/dashboard/seller`	Seller dashboard
`/dashboard/admin`	Admin dashboard
`/pc-builder`	PC builder
`/login`	Login
`/register`	Registration

5.6 Data Fetching Strategy

The frontend uses React Query for all remote state.

Examples:

- `useProducts` for products
- `useListings` for listings
- direct `useQuery` calls in page-level components for cart, checkout, orders, seller analytics, and admin inventory

Data fetching characteristics:

- Fully client-side
- No server-side rendering
- No route-level prefetching
- Query invalidation used after writes

5.7 Frontend State Management

There are three primary Zustand stores.

Store	Purpose
`useUserStore`	User object, JWT token, auth session methods
`usePcBuilderStore`	Selected build part IDs
`useCompareStore`	Compared product IDs

5.8 API Integration Pattern

API communication goes through Axios.

Key file:

- `frontend/src/services/apiClient.ts`

Technical behavior:

- Base URL uses `NEXT_PUBLIC_API_URL`
- Falls back to `http://localhost:8080/api`
- Interceptor adds bearer token from `localStorage`
- All feature modules expect backend envelope format:

```
{
  "success": true,
  "data": {}
}
```

5.9 Frontend Domain Modules

Feature wrappers live under `frontend/src/features`.

Major modules:

- `auth.ts`
- `products.ts`
- `listings.ts`
- `gpus.ts`
- `cart.ts`
- `orders.ts`
- `addresses.ts`
- `paymentMethods.ts`
- `wishlist.ts`
- `sellers.ts`
- `pcBuilder.ts`

These files define:

- Request functions
- Shared TypeScript interfaces
- Payload shapes
- Domain-specific API routes

6. Backend Architecture

6.1 Backend Framework

The backend is a NestJS modular monolith.

Key files:

- `backend/src/main.ts`
- `backend/src/app.module.ts`

Runtime characteristics:

- Global `/api` prefix
- Global validation pipe
- Global response transform interceptor
- Global exception filter
- JWT-based auth guards
- Role-based access control

6.2 Main Backend Modules

Active module graph:

Module	Purpose
`UsersModule`	Registration, login support, current-user lookup
`AuthModule`	JWT strategy and auth service
`ProductsModule`	Product catalog and product price history
`SellersModule`	Seller entities and seller analytics
`ReviewsModule`	Product reviews
`PcBuilderModule`	Compatibility validation
`SearchModule`	Search endpoints and Elastic health
`AddressesModule`	Address management
`PaymentMethodsModule`	Payment method management
`CartModule`	Mixed product/listing cart
`OrdersModule`	Checkout and order persistence
`GpusModule`	Active GPU reference database
`ListingsModule`	Active GPU marketplace listing system

6.3 Request Processing Pipeline

Every backend request follows this pattern:

1. HTTP request enters NestJS
2. Global prefix routes under `/api`

3. Guards run when required
4. ValidationPipe transforms and validates DTOs
5. Controller method forwards to service
6. Service uses TypeORM repositories
7. Response is wrapped by `TransformInterceptor`

Success response shape:

```
{
  "success": true,
  "data": { "...": "..." }
}
```

Error response shape:

```
{
  "success": false,
  "timestamp": "2026-03-12T00:00:00.000Z",
  "path": "/api/example",
  "error": { "...": "..." }
}
```

6.4 Authentication and Authorization

Auth flow:

- Users register or login through `/users`
- `AuthService` signs JWT tokens
- `JwtStrategy` validates bearer tokens
- `JwtAuthGuard` protects private routes
- `RolesGuard` enforces `@Roles(...)`

Role values:

```
USER
SELLER
ADMIN
```

Representative guarded routes:

- Seller/admin create product
- Seller/admin create listing
- Admin seed demo inventory

- Authenticated wishlist, cart, orders, addresses, payment methods

6.5 API Surface

Major route groups:

```
/api/users
/api/products
/api/search
/api/gpus
/api/listings
/api/wishlist
/api/my-listings/analytics
/api/cart
/api/orders
/api/addresses
/api/payment-methods
/api/reviews
/api/sellers
/api/pc-builder
```

6.6 Service Responsibilities

Service	Responsibility
`UsersService`	User creation, lookup by email, lookup by ID
`AuthService`	Credential validation and JWT signing
`ProductsService`	Product CRUD, SQL search, price history, Redis, Elastic indexing
`ListingsService`	GPU listing CRUD, seller profile payloads, analytics, view tracking
`WishlistService`	Save and remove listing wishlists
`MarketplaceUsersService`	Mirror auth users into listing-side `users` table
`SellersService`	Seller rows, approval status, analytics
`CartService`	Mixed inventory cart operations
`OrdersService`	Checkout and order persistence
`AddressesService`	Saved addresses
`PaymentMethodsService`	Saved payment metadata
`ReviewsService`	Product reviews and aggregate rating updates
`GpusService`	GPU reference seed and search
`PcBuilderService`	Build compatibility validation

`SearchService`	Search routing and Elastic health

6.7 Search, Cache, and Indexing Layers

Redis

- Configured through `backend/src/config/redis.config.ts`
- Injected into products module
- Used only for `products:featured`

Elasticsearch

- Configured through `backend/src/config/elastic.config.ts`
- Product documents are indexed on create and update
- Product documents are deleted on product delete
- Search health is exposed through `/search/health`
- Real-time user search still relies on SQL

6.8 Background Work and Startup Logic

There are no queue workers or cron jobs.

The only true startup background-like behavior is:

- `GpusService` implements `OnModuleInit`
- On module startup it checks for the CSV seed file
- It inserts GPU rows with `orIgnore()`

6.9 Legacy and Duplicate Backend Trees

The repository still contains duplicate legacy modules:

- `backend/src/modules/gpus`
- `backend/src/modules/listings`

These are not the active modules imported by `AppModule`, but they still define conflicting entities and services. They are a major source of architectural confusion.

7. Database Design

7.1 Database Runtime Model

The backend uses PostgreSQL through TypeORM.

Key configuration:

- `backend/src/config/configuration.ts`
- `backend/src/config/ormconfig.ts`

Important operational fact:

- `synchronize: true` is enabled

This means the runtime entity model is currently more authoritative than the SQL migration file.

7.2 Core Schema Domains

The database is effectively split into two domains:

1. **Auth / catalog / checkout domain**
2. **GPU marketplace listing domain**

7.3 Auth, Catalog, and Commerce Tables

Representative tables from the entity model:

Table	Purpose
`user`	Auth user record
`seller`	Seller entity tied to `user`
`product`	General product catalog item
`price_history`	Product price history
`review`	Product reviews
`address`	Saved shipping addresses
`payment_method`	Saved payment metadata
`order`	Order header
`order_item`	Order line items
`cart_item`	Mixed inventory cart rows

7.4 GPU Marketplace Tables

Representative marketplace tables:

Table	Purpose
`gpus`	Canonical GPU reference records
`listings`	GPU marketplace listing rows
`listing_images`	Ordered image URLs per listing
`listing_views`	Listing impression tracking
`wishlists`	Listing wishlist saves

`listing_price_history`	Historical listing price changes
`gpu_price_history`	GPU-level or listing-linked price snapshots
`users`	Mirrored marketplace user rows for listing subsystem

7.5 Relationships

Important relationships include:

- `user` -> many `seller`
- `seller` -> many `product`
- `product` -> many `review`
- `product` -> many `price_history`
- `user` -> many `address`
- `user` -> many `payment_method`
- `user` -> many `order`
- `order` -> many `order_item`
- `user` -> many `cart_item`
- `gpus` -> many `listings`
- `users` -> many `listings`
- `listings` -> many `listing_images`
- `listings` -> many `listing_views`
- `users` -> many `wishlists`
- `listings` -> many `wishlists`

7.6 Indexes

Explicit indexes observed in the active entity model:

Index	Purpose
`IDX_gpus_slug`	Unique GPU slug
`IDX_gpus_manufacturer`	GPU manufacturer filtering
`IDX_gpus_vram_gb`	GPU VRAM filtering
`IDX_gpu_price_history_gpu_recorded_at`	Price history lookup by GPU and time
`IDX_listing_images_listing_sort`	Ordered listing image retrieval
`IDX_wishlists_user_listing`	Enforce one wishlist save per user and listing

7.7 Seed Data

Current seed mechanisms:

- **GPU CSV seed**

- `backend/database/seed/gpu_seed_series.csv`
- Applied on startup by `GpusService`
- Also available through `npm run seed:gpus`

- **Demo product seed**
  - Generated in `ProductsService`
  - Triggered by admin API
- **Demo listing seed**
  - Generated in `ListingsService`
  - Triggered by admin API

7.8 Schema Drift and Migration Reality

The SQL migration file is:

- `database/migrations/001_init.sql`

Current issue:

- It does not fully match the active runtime entity model
- It omits active marketplace tables such as `gpus`, `listing_images`, `listing_views`, `wishlists`, `listing_price_history`, and `gpu_price_history`
- It references `listings` from `cart_item` without fully defining the active listing schema

Conclusion:

- The running schema currently depends on TypeORM synchronization, not reliable migrations-first discipline

8. Hardware Reference Database

8.1 Purpose

The hardware reference system is designed to normalize listings against canonical hardware data instead of relying only on seller-entered free text.

8.2 Current Scope

The only fully implemented reference database is the GPU dataset.

There is **no equivalent CPU, motherboard, RAM, or storage reference database** in the current codebase.

8.3 GPU Reference Data Model

The `Gpu` entity stores:

- `id`
- `name`

- `slug`
- `manufacturer`
- `architecture`
- `releaseYear`
- `processNm`
- `vramGb`
- `memoryType`
- `memoryBusWidth`
- `pcieVersion`
- `tdpWatts`

8.4 Slug System

The slug is a unique indexed identifier.

Examples from the seed file:

```
arc-a310
arc-a770
```

The slug is used to:

- Normalize search
- Avoid freeform title drift
- Expose consistent canonical GPU identities

8.5 Listing Normalization Flow

GPU listing creation flow:

1. Seller searches reference records using `/search/gpus`
2. Frontend presents canonical GPU search results
3. Seller chooses a GPU reference row
4. Listing stores `gpu_id`
5. Listing detail page renders canonical GPU specs from the reference row

8.6 Search Accuracy Benefits

This improves marketplace quality in several ways:

- Buyer search is not dependent on inconsistent seller naming
- Listing cards use standardized GPU names
- Detail pages show consistent architecture, VRAM, PCIe, and TDP data
- Search operates on canonical GPU fields instead of only seller prose

8.7 Price Modeling Tied to Reference Data

Demo listing generation uses GPU reference attributes to estimate seeded listing prices.

The price heuristic uses:

- Release year
- VRAM
- Memory bus width
- TDP
- Manufacturer

This is not live pricing intelligence. It is a demo-seeding heuristic that produces plausible marketplace examples.

9. Infrastructure

9.1 Container Stack

The platform is orchestrated with Docker Compose.

Primary file:

- `docker-compose.yml`

9.2 Runtime Services

Service	Technology	Purpose	Exposed Port
`postgres`	PostgreSQL 16 Alpine	Primary relational database	`5432`
`redis`	Redis 7 Alpine	Cache	`6379`
`elasticsearch`	Elasticsearch 8.13	Product indexing and health endpoint	`9200`
`backend`	NestJS app	API server	internal `4000`
`frontend`	Next.js app	Web UI	internal `3000`
`nginx`	Nginx	Reverse proxy	`8080`

9.3 Reverse Proxy Topology

Nginx routes traffic as follows:

```
/api/* -> backend:4000
/*     -> frontend:3000
```

Key file:

- `nginx/default.conf`

9.4 Environment Variables

Current runtime configuration is sourced from compose plus backend config defaults.

Representative environment variables:

```
PORT=4000
DB_HOST=postgres
DB_PORT=5432
DB_USER=pc_user
DB_PASS=pc_pass
DB_NAME=pc_hardware
REDIS_HOST=redis
REDIS_PORT=6379
ES_NODE=http://elasticsearch:9200
ES_INDEX=products
JWT_SECRET=supersecretjwt
NEXT_PUBLIC_API_URL=/api
```

9.5 Volumes and Persistence

Current persistent storage:

- PostgreSQL data volume: `pgdata`

Current non-persistent infrastructure data:

- Redis data
- Elasticsearch data

9.6 Development and Production Characteristics

The project does not currently have a strong separation between development and production configuration.

Observations:

- Docker Compose is the main environment model
- Elasticsearch security is disabled
- JWT secret has a hardcoded default fallback
- No secret-management layer exists
- No infrastructure-as-code beyond Docker Compose and a basic CI workflow

9.7 CI Workflow

The repository includes:

- `.github/workflows/ci.yml`

Current CI steps:

- Checkout
- Install backend dependencies
- Install frontend dependencies
- Lint backend
- Lint frontend
- Test backend
- Test frontend
- Build backend
- Build frontend
- Build Docker images
- Placeholder deploy step

Operational reality:

- The repo contains effectively no real test suite
- The deploy step is only a placeholder

10. Current Limitations

10.1 Product and Marketplace Model Split

The application presents one marketplace experience in the UI, but the backend still uses two inventory models:

- `Product`
- `Listing`

This causes inconsistent behavior across:

- Seller analytics
- Seller profile visibility
- Wishlist support
- Compare support
- Price tracking
- Edit and delete flows
- Builder data sources

10.2 GPU-First Normalization

The strongest normalization only exists for GPUs.

What is missing:

- CPU reference records
- Motherboard reference records
- RAM reference records
- Storage reference records

- PSU reference records

10.3 Search Inconsistency

Search is described as Elasticsearch-powered, but actual browse search remains primarily SQL-backed. This creates a mismatch between infrastructure and delivered product behavior.

10.4 Incomplete Seller Experience

Seller flows are incomplete because:

- Seller approval UI is missing
- Product sellers do not have full product-management screens
- Seller dashboard metrics are placeholders
- Listing analytics and seller analytics are split

10.5 Incomplete Buyer Experience

Buyer-facing gaps include:

- No contact seller or messaging feature
- No negotiation or offer flow
- No listing price chart on listing detail page
- No product wishlist
- No review UI

10.6 Incomplete Commerce Layer

Commerce is only partially real:

- Orders persist
- Checkout exists
- Payment methods can be saved

But:

- Payments are not processed
- Inventory is not reserved
- Fulfillment workflows are absent
- Order state transitions are not actively managed

10.7 Migration and Deployment Readiness

The system is not migrations-first and depends on `synchronize: true`, which is not safe for production evolution.

11. Technical Debt

11.1 High-Severity Debt

- Runtime schema synchronization is enabled in production-style runtime configuration.
- The migration SQL does not represent the actual active schema.
- Checkout is not transactional and has no inventory reservation.
- Backend module duplication creates ambiguous source-of-truth boundaries.

11.2 Structural Debt

- `Product` and `Listing` are overlapping inventory models.
- `Seller` and `MarketplaceUser` are overlapping seller identities.
- Auth `user` and marketplace `users` are separate tables with mirrored data.

11.3 Search and Performance Debt

- Search relies on ILIKE-heavy SQL across text and JSON fields.
- Listing search and product search use different semantics.
- No pagination or large-catalog browsing strategy is implemented.
- Elasticsearch infrastructure is underused relative to its operational cost.

11.4 Type Safety and Code Quality Debt

- Backend TypeScript is configured with `strict: false`.
- Several services rely on `any`.
- `ReviewsService` reaches into another service's repository internals to update ratings.

11.5 Frontend Debt

- All data-heavy pages are client-rendered only.
- State invalidation and fetch logic are duplicated across many pages.
- Route aliases create user-facing simplicity but add code-path duplication.

11.6 Product and Copy Debt

- Several pages still contain placeholder or roadmap-style copy.
- Demo inventory is central to the current experience in many routes.
- The deals page is not behaviorally accurate to its label.

12. MVP Evaluation

12.1 Essential for the MVP

Keep and Strengthen	Why It Matters
Unified marketplace browsing	This is the primary user value proposition
GPU reference normalization	It materially improves search quality and listing consistency
Listing and product detail pages	Buyers need trustworthy item detail views

Authentication and role gating	Sellers and buyers need protected account flows
Cart and order persistence	Core commerce journey
Seller profile and seller inventory management	Buyers need seller context, sellers need control
Search and filtering	Essential for marketplace usability
Address management	Necessary for checkout

12.2 Simplify or Remove for Early MVP Focus

Simplify or Defer	Reason
Separate `Product` and `Listing` models	Marketplace-first MVP should converge on one inventory model
Deals page	Adds little value without real discount logic
Saved payment methods complexity	Can be simplified until real payments exist
Elasticsearch	Can be deferred until search volume and query complexity justify it
Redis beyond minimal use	Current usage does not justify broader complexity
Admin panel breadth	Keep only the operational actions needed to launch
Advanced seller approval workflow	Only keep if moderation is truly enforced
Overly broad category support without reference data	Better to be strong on fewer categories than weak on many

12.3 Strategic MVP Recommendation

The current best MVP path is:

1. Standardize on one marketplace inventory model
2. Keep GPU normalization as the reference architecture
3. Expand normalization to the next highest-value component classes only when needed
4. Harden browse, search, seller pages, cart, and checkout before expanding secondary tools

13. Full System Diagram

13.1 Layered System Diagram

```
User
  -> Browser
  -> Next.js Frontend
     -> Pages Router
     -> React Query
     -> Zustand Stores
     -> Axios API Client
  -> Nginx Reverse Proxy
```

```
  -> NestJS API
     -> Guards
     -> Controllers
     -> Services
     -> TypeORM Repositories
  -> PostgreSQL
  -> Redis
  -> Elasticsearch
```

13.2 Request and Data Flow

```
User Action
  -> Frontend Page or Component
  -> Feature API Wrapper
  -> Axios Request to /api
  -> Nginx
  -> NestJS Controller
  -> Service Layer
  -> Database / Cache / Search
  -> Service Response
  -> Global Response Envelope
  -> Frontend React Query Cache
  -> Rendered UI
```

13.3 Layer Responsibilities

Layer	Responsibility
User	Initiates browse, search, buy, and sell actions
Frontend	Renders UI, manages client state, triggers API requests
API Client	Sends authenticated requests to backend
Nginx	Routes frontend and API traffic
NestJS Controllers	Map HTTP routes to services
NestJS Services	Execute business logic
TypeORM	Persistence abstraction over PostgreSQL
PostgreSQL	Stores platform data
Redis	Caches latest product list
Elasticsearch	Stores indexed product documents and health state

14. Conclusion

PCForge is already a meaningful full-stack marketplace application rather than a static prototype. The codebase contains:

- A real frontend
- A real API
- Real persistence
- Real buyer flows
- Real seller entry points
- Real inventory data structures
- Real infrastructure containers

Its strongest implemented area is the **GPU marketplace**, where canonical hardware reference data improves listing quality, detail pages are useful, seller profiles exist, and buyer actions such as wishlist, cart, and checkout are connected.

Its weakest architectural area is the **split data model**, where general `Product` records and GPU `Listing` records overlap without being fully unified. That split drives much of the current inconsistency in seller workflows, analytics, price history, search behavior, and buyer expectations.

In its current state, the platform should be understood as:

- A strong GPU-first used hardware marketplace foundation
- A partially generalized hardware catalog
- A working but incomplete commerce flow
- A codebase with real MVP potential, but clear consolidation and production-hardening work still required

For engineering, the next priority is not feature expansion first. It is model unification, schema discipline, and buyer/seller flow consistency.